Laboratorio Integrador – Sistema de Observabilidad end-to-end con AIOps y Resiliencia — Implementación en AWS

Danny Tatiana Morales Jiménez
Juan Daniel Valderrama Pérez
Camilo Mora Rodriguez

Observabilidad en Ambientes Productivos
María Fernanda Ochoa Paipilla
Octubre 2026



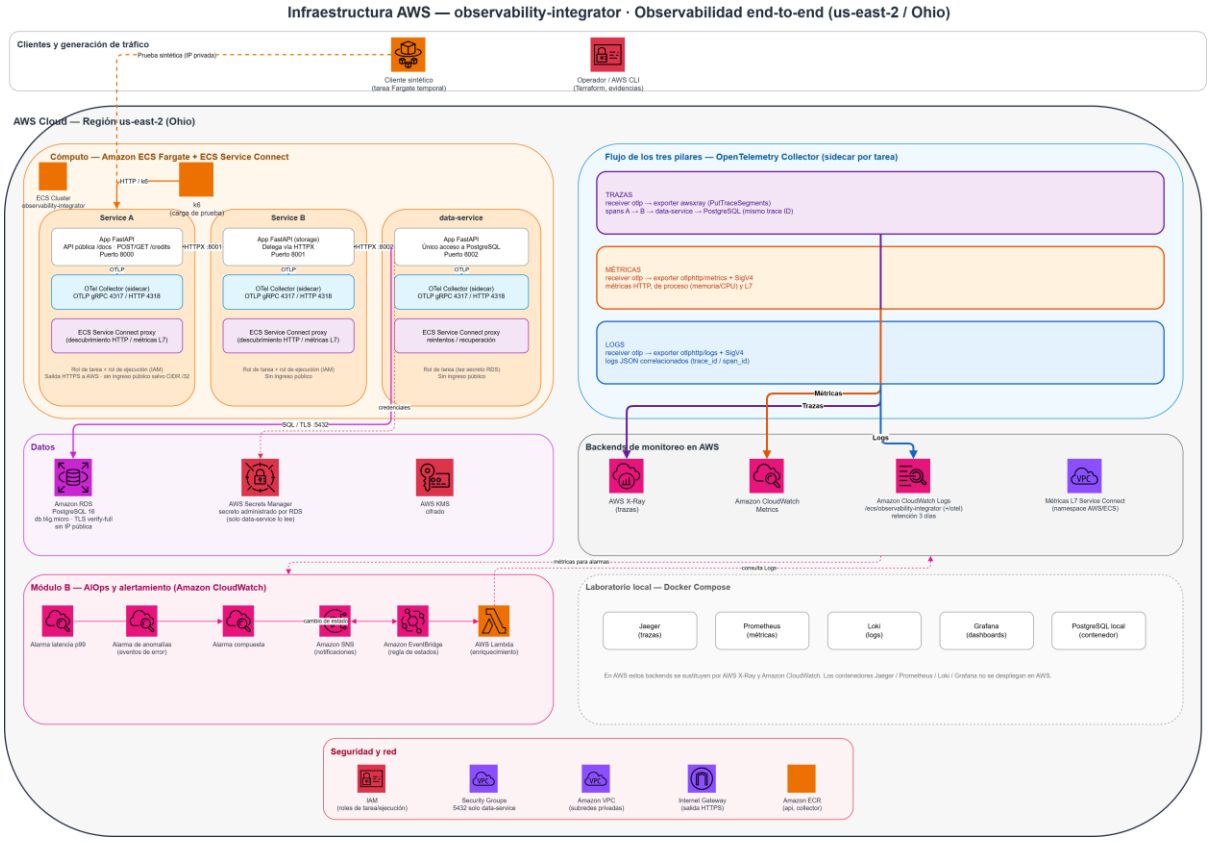
Arquitectura Observable Completa

Repositorio GitHub: GitHub - Laboratorio Integrador

Video Explicativo de la Infraestructura en AWS: Video Explicativo

Implementamos y validamos en AWS el recorrido Service A → Service B → data-service → PostgreSQL RDS. Con OpenTelemetry seguimos solicitudes mediante trazas, métricas y logs. Configuramos componentes de AIOps y comprobamos registros y alertamiento de red. Ejecutamos dos experimentos de caos y medimos su impacto en el objetivo de servicio (SLO) y el presupuesto de errores. La correlación AIOps completa y el tiempo medio de detección (MTTD) inferior a dos minutos quedaron pendientes. Usamos ECS Service Connect; no lo presentamos como cumplimiento literal de AWS App Mesh. La Figura 1 muestra nuestra arquitectura y las tres señales. Cerramos con una autoevaluación de madurez y un roadmap de tres meses.

Figura 1. *Diagrama Arquitectónico de Observabilidad.*



AIOps: Detección Automática de Anomalías

Para AIOps adoptamos el criterio tasa de errores > línea base + 2σ Y latencia p99 > 1000 ms, en el mismo servicio y periodo. La línea base describe el comportamiento habitual y σ su variación. Configuramos DataServiceErrorEvents en Observability/ModuleB, una alarma de anomalías ANOMALY_DETECTION_BAND de amplitud 2, una alarma p99 sobre TargetResponseTime de AWS/ECS y una alarma compuesta con condición AND. DataServiceErrorEvents cuenta eventos de logs: no es una tasa de solicitudes fallidas. La banda de amplitud 2 no demuestra por sí sola la fórmula requerida. También debemos validar ventanas comparables: la alarma p99 usa 60 s y la de anomalías 300 s.

Integramos EventBridge, Lambda y SNS para consultar logs y enriquecer notificaciones. Documentamos pruebas individuales de SNS y una invocación sintética de Lambda. No demostramos la cadena automática completa ante un incidente real ni una alerta enriquecida con su trace_id. La correlación y la reducción de ruido son objetivos pendientes, no resultados medidos. La Tabla 1 distingue configuración y evidencia.

Tabla 1. Configuración de alertamiento y límites de la evidencia.

Criterio	Umbral estático	AIOps correlacionado	Evidencia
Lógica de disparo	Regla única: alerta si una señal cruza un valor fijo	Alarma compuesta AND; simultaneidad de las mediciones pendiente de validar	infra/aws/metrics-queries.md §Condición conjunta
Señales evaluadas	1 (conteo de errores *o* latencia, de forma aislada)	2 (DataServiceErrorEvents **y P**99 TargetResponseTime)	Módulo B, secciones 3.2–3.3; referencia al final
Criterio de error	Un evento aporta 1 al contador; no demostramos una comparación estática equivalente	Banda sobre conteo de eventos, amplitud 2; no equivale a una tasa > línea base + 2σ validada	Módulo B, secciones 3.2–3.3; referencia al final

Criterio	Umbral estático	AIOps correlacionado	Evidencia
Criterio de latencia	*P*99>1000 ms (fijo)	*P*99>1000 ms (SLO, como condición necesaria)	Módulo B, secciones 3.2–3.3; referencia al final
Ventana de evaluación	p. ej. 60 s / 1 periodo	p99: 60 s, 2/2 puntos; anomalía: 300 s, 2/3 puntos. Falta alinear ventanas	Módulo B, secciones 3.2–3.3; referencia al final
Exp. 1 — latencia +200 ms, sin errores	p99 cliente bajo 1000 ms; sin evidencia de una alarma durante el experimento	No demostramos detección conjunta; el p99 cliente no valida la serie del servidor	docs/CHAOS_EXPERIMENT_1.md; consumo SLO cliente = **0 %**
Exp. 2 — 10 % errores internos recuperados	50 errores internos observados; no medimos cuántas alertas estáticas habrían generado	**No demostramos una alerta correlacionada ni su comparación con reglas estáticas**	Informe del experimento 2; referencia al final
Alertas ruidosas (ventana de 180 s)	No medimos una línea comparativa de alertas	**Reducción de ruido no medida**	Pendiente: misma carga, ventana y definición de ruido

Diagrama de Flujo de la Regla de Correlación: Online FlowChart - Diagrama de Flujo

Network and Security Observability

Configuramos en AWS la cadena privada A → B → data-service → RDS. Los Security Groups limitan los puertos 8001, 8002 y 5432 a los servicios autorizados. RDS no tiene IP pública y usa TLS con verificación del certificado. Las tareas usan roles IAM temporales; solo data-service

recibe el secreto de RDS. No empleamos NAT Gateway ni ALB y conservamos cerrada la entrada HTTP pública. Las solicitudes y trazas demuestran la comunicación funcional, pero no prueban todas las denegaciones de red o permisos.

Comprobamos VPC Flow Logs activos, registros ACCEPT/REJECT, un filtro métrico, una alarma con acción SNS y un dashboard. El historial muestra la acción SNS y el informe del módulo C registra recepción del correo. La alarma cuenta rechazos; no demuestra por sí sola un ataque. Inspector y Security Hub están reportados como bloqueados por políticas de organización. Faltan una línea base y fuentes verificadas de autenticaciones y CVEs. Conservamos como riesgos los tres hallazgos HIGH de api:initial y la limitación de escaneo del Collector; una imagen nueva no demuestra su corrección.

Figura 2. *Diagrama de Arquitectura de Red y Seguridad.*

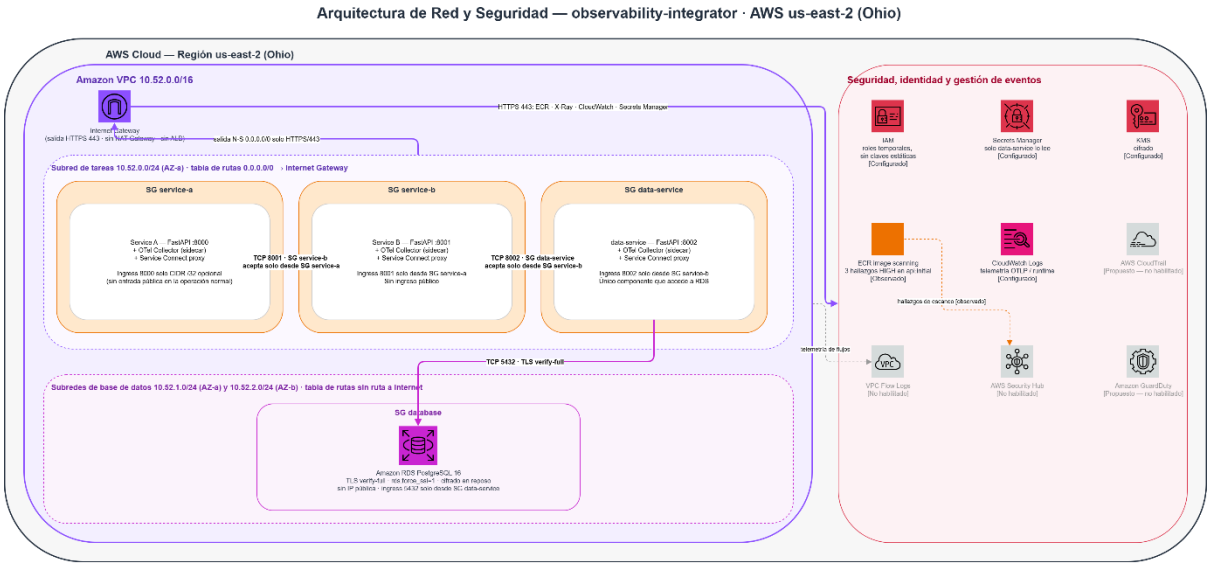


Tabla 2. *Estrategia de Network and Security Observability.*

Dominio de Seguridad / Red	**Métrica o Indicador Clave (Golden Signal)**	**Fuente de Telemetría / Herramienta**	**Criterio de Alerta / Umbral Configurado**
Tráfico de red (N-S y E-W)	Flujos aceptados/rechazados entre microservicios y hacia Internet;	[Comprobado] VPC Flow Logs → CloudWatch Logs, registros ACCEPT/REJECT, filtro RejectedFlowRecords y	[Comprobado] Alarma por ≥50 registros REJECT en 300 s, transición a ALARM y acción SNS. [Pendiente] Línea base y validación

Dominio de Seguridad / Red	**Métrica o Indicador Clave (Golden Signal)**	**Fuente de Telemetría / Herramienta**	**Criterio de Alerta / Umbral Configurado**
	latencia y errores L7 por salto	dashboard. [Configurado] Security Groups y métricas L7 de Service Connect.	de un evento controlado; un rechazo no demuestra un ataque.
Control de acceso / Autenticación	Uso de identidades temporales, lecturas del secreto de base de datos y tasa de autenticaciones fallidas	[Configurado] IAM (roles de tarea/ejecución, políticas inline, sin claves estáticas) · Secrets Manager (solo data-service) · [Propuesto] CloudTrail/CloudWatch Logs para eventos AccessDenied; el laboratorio no expone autenticación de usuario en service-a	[Pendiente] Fuente fiable de fallos de autenticación y detección controlada. Service A no expone autenticación de usuario. [Configurado] Roles temporales y secreto solo para data-service.
Gestión de vulnerabilidades	Inventario de CVEs activos por imagen y hallazgos de severidad alta (CVSS ≥ 7.0)	[Observado] Escaneo ECR e informe histórico de hallazgos. [Limitación reportada] Inspector y Security Hub bloqueados por políticas de organización.	[Observado] Tres HIGH en api:initial y limitación de escaneo del Collector. [Pendiente] Inventario actualizado de CVEs, remediación y alertamiento; no afirmamos ausencia de vulnerabilidades.

Distinguimos resultados comprobados, configuraciones y pendientes; conservamos las capturas del módulo C como respaldo.

Chaos Engineering Controlado

Ejecutamos dos fallos controlados, reversibles y limitados a solicitudes sintéticas. Usamos app/chaos.py y app/error_chaos.py, desactivados por defecto y con caducidad máxima de 30 minutos. Para la latencia exigimos una cabecera; para los errores, un UUID sintético y un marcador de trace_id. Excluimos los healthchecks y las escrituras de la inyección de errores. Registramos cada inyección mediante spans, logs correlacionados y métricas. La Tabla 3 conserva las cifras originales y distingue el SLO del cliente del servidor.

Localizamos los servicios afectados en trazas A → B → data-service → PostgreSQL y calculamos el impacto en el SLO y el presupuesto de errores. No demostramos una alerta automática correlacionada: el MTTD quedó sin medir y no acreditamos la meta inferior a dos minutos. Los tiempos de observación de logs son comprobaciones operativas, no notificaciones. Conservamos también el intento inicial inválido del experimento 2 y su diagnóstico en el repositorio.

Tabla 3. *Resultados de los experimentos de caos.*

Experimento de Caos	**Vector de Fallo Inyectado**	**MTTD Registrado (Objetivo: < 2 min)**	**Impacto en SLO y Error Budget**	**Evaluación de Accionabilidad**
Experimento 1 — Latencia en service-b	+200 ms por solicitud en B (span chaos.latency), 180 s por fase; gate: CHAOS_ENABLED + cabecera x-lab-chaos-experiment + expiración ≤ 30 min	Referencia operativa: primera verificación del controlador 29,58 s tras el primer log de B, es sondeo de logs, no MTTD de alertamiento	Consumo del error budget: 0 % (SLO de trabajo 99,5 % / ≤1000 ms). 540 solicitudes por fase, cero errores; p99 por fase 72,00 → 276,58 → 84,18 ms, siempre bajo el umbral de 1000 ms. Sin activación de la	Sí. La traza OTel/X-Ray permitió identificar a service-b como cuello de botella: mismo trace_id en A → B → data-service → PostgreSQL y span chaos.latency. No hubo alerta real correlacionada.

Experimento de Caos	Vector de Fallo Inyectado	MTTD Registrado (Objetivo: < 2 min)	Impacto en SLO y Error Budget	Evaluación de Accionabilidad
			condición conjunta	
Experimento 2 — 10 % de errores en data-service	HTTP 503 en 1 de cada 10 intentos físicos elegibles (span chaos.error), 180 s por fase; gate: DATA_CHAOS_* + UUID sintético + marcador de trace_id	Referencia operativa: primer fallo real verificado en logs demora 32,188 s; el sondeo del controlador no es una notificación del sistema AIOps	Doble denominador: servidor 50/500 intentos elegibles = 10,00 %, con burn rate 20× y consumo de ventana 2000 % del SLO de disponibilidad del servidor; cliente 0 % (1350/1350 respuestas finales 200 en las tres fases, recuperadas por reintentos de Service Connect)	Parcial. 12 trazas X-Ray validaron la cadena padre-hijo y el intento 503 → reintento 200 con SQL, unidos por trace_id. No demostramos una alerta real enlazada. Por separado, la prueba sintética de Lambda registró trace_id_found = false.

Diagrama de Secuencia de Los Experimentos del Caos: Online FlowChart - Secuencia Experimentos

Madurez y roadmap de tres meses

Usamos los ocho componentes de la presentación del curso, página 40. Las notas de 1 a 5 son nuestra autoevaluación provisional, no una escala oficial. Fundamentamos la matriz en las evidencias y limitaciones del repositorio.

Componente	Nivel / 5
Observability	3
The Three Pillars	3
OpenTelemetry	3
Time Travel Topology	2
Network and Security Observability	2
AIOps	2
DataOps	1
SRE, DevOps and Observability	2

Roadmap conjunto: mes 1, cotejamos las 8 fuentes y notas, completamos el inventario de AIOps/seguridad y validamos solicitudes, errores y p99 en 3/3 servicios. Mes 2, probamos los cuatro casos de la condición Y y al menos tres detecciones comparables, con meta ≤120 s aún pendiente; validamos un evento de red y sus fuentes. Mes 3, completamos al menos tres repeticiones comparables por mecanismo, una recuperación aislada aprobada y la revisión de los 8 componentes. Son metas propuestas, no resultados alcanzados. Conservamos responsables conjuntos, prioridades e indicadores completos en la evaluación enlazada.

Conclusiones

Comprobamos en AWS la comunicación de tres microservicios con RDS mediante trazas, métricas y logs. Los experimentos permitieron identificar retrasos y errores internos, verificar la recuperación por reintentos y medir su impacto. Conservamos las evidencias y eliminamos los recursos temporales. Estas pruebas no representan una validación en producción.

En AIOps validamos componentes individuales, pero no la correlación completa ni la reducción de ruido. En seguridad comprobamos Flow Logs, alarmas, SNS y dashboard, aunque persisten limitaciones. No medimos el MTTD ni demostramos la meta de dos minutos. Nuestro roadmap busca cerrar estas brechas y mejorar la madurez.

Referencias

Grafana Labs. (s. f.-a). *Grafana k6 documentation*. https://grafana.com/docs/k6/latest/

Grafana Labs. (s. f.-b). *Trace integrations*. https://grafana.com/docs/grafana/latest/explore/trace-integration/

Jaeger. (s. f.). *Architecture*. https://www.jaegertracing.io/docs/architecture/

OpenTelemetry. (s. f.). *OpenTelemetry Python documentation.* https://opentelemetry-python.readthedocs.io/

Observabilidad MAS 2026-5. (2026a). *Evaluación de madurez y roadmap de observabilidad [Documento del repositorio]*. GitHub. https://github.com/observabilidad-mas-2026-5/observability-integrator/blob/main/docs/EVALUACION_MADUREZ_ROADMAP.md

Observabilidad MAS 2026-5. (2026b). *Laboratorio integrador de observabilidad: código, infraestructura y evidencias [Repositorio de código]*. GitHub. https://github.com/observabilidad-mas-2026-5/observability-integrator

Universidad de La Sabana. (2026). *Presentación del curso de Observabilidad en Ambientes Productivos [Diapositivas de clase]*. Material académico.

World Wide Web Consortium. (2021*). Trace context*. https://www.w3.org/TR/trace-context/